Week 4: Attention

Reading

- Online normalizer calculation for softmax.
- FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. Pages 1-5 only.
- FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. Pages 1-6 only.
    - Enhancements over V1:
        - **This week: Restructuring of the core Algorithm**
        - Next week: Use tensor cores
        - Next week: Use warp-level work partitioning
- [Optional] How to read a paper.
- [Optional] How to Read an Engineering Research Paper

Note: FlashAttentionV1 paper is good to read for background, but the algorithm written in the paper is awkward to implement. The Algorithm 1 in FlashAttentionV2 has a cleaner, natural structure.

Thus, we will focus our initial implementation efforts on Algorithm 1 of FlashAttentionV2. We wil
move on to the more advanced aspects of FlashAttentionV2 over the next couple weeks (tenso
cores, work partitioning, etc).

Assignment

1. Using *unparallelized C*, implement Algorithm 1 in Section 3.1 of the FlashAttention-2 paper.
2. Using *parallelized CUDA*, implement Algorithm 1 in Section 3.1 of the FlashAttention-2 pape

The goal of this assignment a *correct, parallelized implementation of FlashAttention*. Don't
worry about lower level optimization for now (e.g. tensor cores, memory coalescing, etc.). We'll
address those later.

Discussion

Definition of attention:

$$O = Attention(Q, K, V) = softmax(QK^T / \quad d)V$$

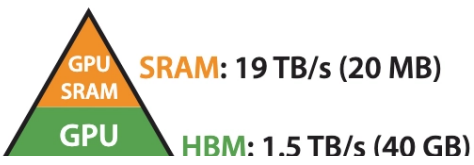
$$O, Q, K, V \in \mathbb{R}^{N \times d}$$

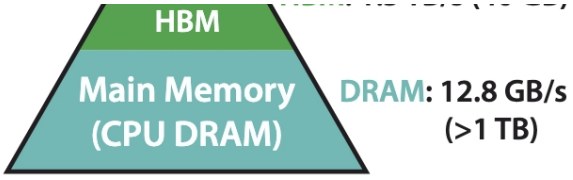
Mathematical operations:

- Matrix multiplication
- Transpose
- Scalar multiplication of a matrix
- Softmax

Definition of Softmax:

$$y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}}$$





Memory Hierarchy with Bandwidth & Memory Size

FlashAttention takes the shared memory tiling idea for matrix multiplication and applies to attention by leveraging the online softmax trick.

Key correctnes property (similar to matrix multiplication): Each threadblock writes a different part of the output matrix O in the HBM.

Approach:

- Divide N into B_r-sized chunks.
- Each input chunk of Q_i of Q is assigned to a threadblock where $Q_i \in \mathbb{R}^{B_r \times d}$
  - This threadblock is responsible for the output chunk O_i that corresponds to Q_i.
  - This threadblock iterates over every chunk of K, V.
  - Thus, every threadblock is going to read every chunk of K, V, but only one chunk of Q.

chunk = block = tile

How do we calculate B_r and B_c which are inputs into flash attention?

- These two parameters are constrained by the amount of shared memory available in each threadblock (less than 1MB).

Line 3 launches a kernel with the T_r threadblocks.

Line 4 requires a single Q_i in shared memory.

- Total of $B_r \times d \times 4$ bytes

Line 5 requires a single O_i in shared memory.

- Total of $B_r \times d \times 4$ bytes

Line 7 requires a single K_j in shared memory and a single V_j in shared memory.

- $B_c \times d \times 4$ bytes + $B_c \times d \times 4$ bytes
- Total of $2 \times B_c \times d \times 4$ bytes

Thus, we have a constraint that:

- $B_r \times d \times 4$ bytes + $B_r \times d \times 4 + 2 \times B_c \times d \times 4$ bytes must be less than size of share memory in a single threadblock - the size of the memory for registers.

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
for $1 \le i \le T_r$ **do**
  Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
  On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
  for $1 \le j \le T_c$ **do**
    Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
  end for
  On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
  On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
  Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.
  Write L_i to HBM as the i-th block of L.
end for
Return the output $\mathbf{O}$ and the logsumexp L.

$diag(...)$

$diag(X)$ turns a vector X with shape $(m, 1)$ into a diagonal matrix of shape (m, m) where each matrix element on the diagonal at (i, i) is set to the value $X[i]$.

Thus, $diag(X)M$ scales each row of the matrix M by the corresponding element in X.

This is a common trick in linear algebra to apply element-wise row operations using matrix notation.

$diag(...)^{-1}$

To understand what $Z = diag(X)^{-1}Y$ does, see its implementation below in function `diag_inv_scale`.

```
// X has shape (m, 1) // Y has shape (m, n) // Z has shape (m, n) void
diag_inv_scale(float* X, float* Y, float* Z, int m, int n) { for (int i = 0; i <
m; ++i) { float inv_xi = 1.0f / x[i]; for (int j = 0; j < n; ++j) { Z[i * n + j]
= Y[i * n + j] * inv_xi; } } }
```

Example:

```
X = [2, 4, 5] Y = [ [10, 20], [12, 8], [5, 15] ]
```

Then output will be:

```
[ [10/2, 20/2] = [5.0, 10.0], [12/4, 8/4] = [3.0, 2.0], [5/5, 15/5] = [1.0, 3.0]
]
```

Online Softmax

Lines 10-12 in the flash attention algorithm above implement online softmax. See the paper Online normalizer calculation for softmax, Sections 1-4

$$y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}} \tag{2}$$

Algorithm 3 Safe softmax with online normalizer calculation

1: $m_0 \leftarrow -\infty$

1: $m_0 \leftarrow -\infty$
2: $d_0 \leftarrow 0$
3: **for** $j \leftarrow 1, V$ **do**
4: $\quad m_j \leftarrow \max(m_{j-1}, x_j)$
5: $\quad d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$
6: **end for**
7: **for** $i \leftarrow 1, V$ **do**
8: $\quad y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$
9: **end for**

Calculating B_c, B_r from M

```
typedef struct { tile Kj; // (B_c, d) tile Vj; // (B_c, d) tile Qi; // (B_r, d),
tile Oi; // (B_r, d) tile li; // (B_r, 1) tile mi; // (B_r, 1) tile Sij; //
(B_r, B_c) tile mij; // (B_r, 1) tile Pij; // (B_r, B_c) tile lij; // (B_r, 1)
tile mi_new; // (B_r, 1) tile li_new; // (B_r, 1) } SharedMem;
```

Using the above struct, we derive an inequality between M, B_c, B_r:

$$M < 2 * B_c * d + 2 * B_r * d + 6 * B_r + 2 * B_r * B_c$$

FlashAttentionV2: Algorithm 1